

Introductory Security Fundamentals

CSCE/SOFT 469/869

Fall 2025

Principle: Be reluctant to trust

Source: Building Secure Software: How to Avoid Security Problems the Right Way by John Viega, Gary R. McGraw

- Software issues aside, people “trusting” people is the biggest problem.
 - Scarcely a day goes by without reading of another person being scammed out of money.
 - It’s old-school *and* high-tech confidence games
 - People aren’t concerned enough with their social media data, combined with breaches of data brokers’ information and public records: these all provide information to be used against (potentially lonely) victims.
 - Criminals also use fear, uncertainty and doubt (FUD) to stop people from taking the time to think, “does this even make sense?”
 - One of the common technical attacks is to get the victim to download/install a “helpful” remote access trojan (RAT).

Assume all other endpoints, networks, software and data are suspect

- Counter to unsecure things: “trust, but verify.”
 - Note that this must be paired with the assessed risk/triage or you’ll miss critical/important things or overspend on other things.
- Examples of technical security controls to mitigate risk:
 - Strong authentication,
 - Verifying all inputs,
 - Good-enough authorization, based on assessed risk.

More than another “flavor-of-the-month”: Zero Trust (ZT)

NIST SP800-207: Zero Trust Architecture

- “Zero trust (ZT) is the term for an evolving set of cybersecurity paradigms that move defenses from static, network-based perimeters to focus on users, assets, and resources.”
- “Zero trust assumes there is no implicit trust granted to assets or user accounts based solely on their physical or network location or based on asset ownership.”
- Authentication and authorization are discrete functions performed before a session to a resource is established.

NIST Special Publication 800-207

Zero Trust Architecture

Scott Rose
Oliver Borchert
Stu Mitchell
Sean Connelly

This publication is available free of charge from:
<https://doi.org/10.6028/NIST.SP.800-207>

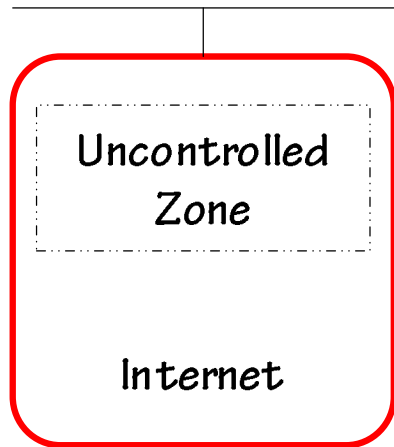
C O M P U T E R S E C U R I T Y

NIST
National Institute of
Standards and Technology
U.S. Department of Commerce

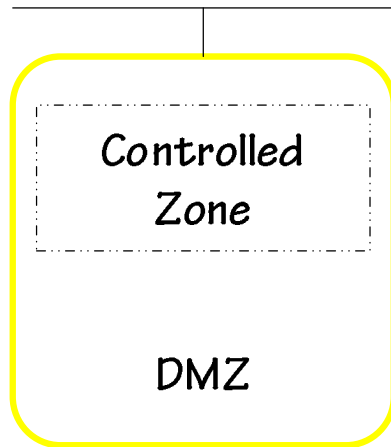
Best practice: Network Segmentation

NIST SP800-215: Guide to a Secure Enterprise Network Landscape

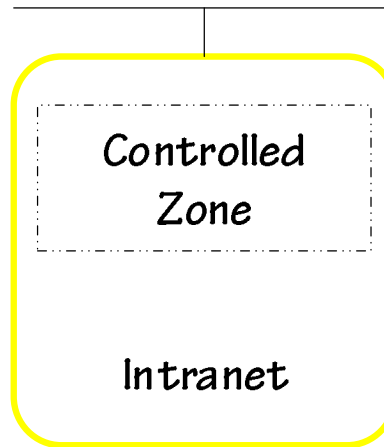
No components should be deployed in an uncontrolled network. It's also generally unsafe for components to communicate with one another across an uncontrolled network.



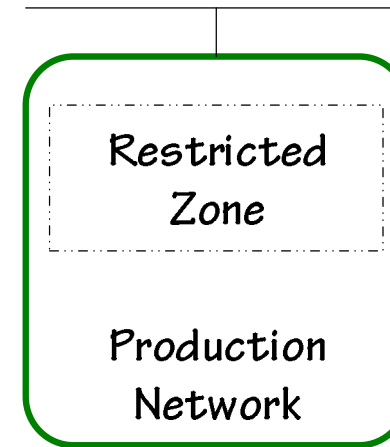
Possible location for GUI servers that face external clients.



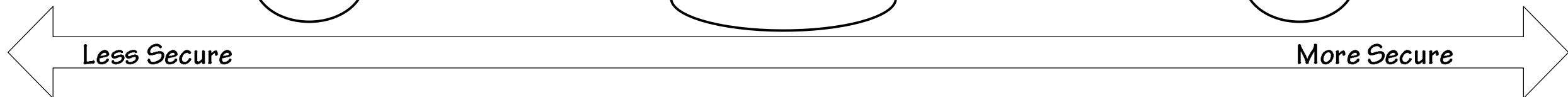
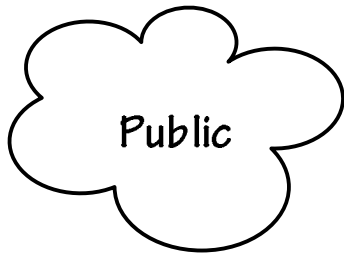
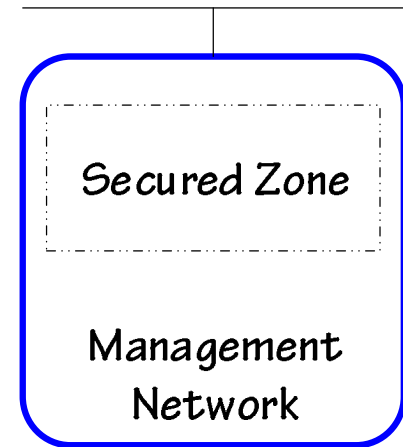
The specific level of trust in an internal network dictates the set of components that may be deployed within them.



Organizations may set up specialized restricted zones for production systems



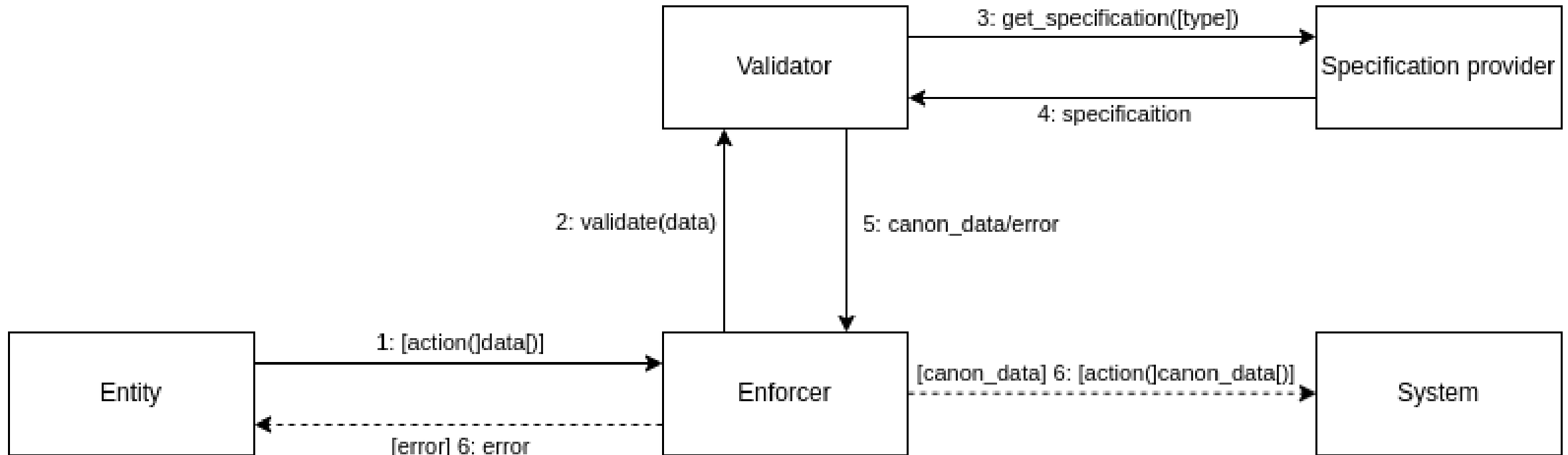
Some organizations set up special networks to separate management components from productions systems and less trusted network zones.



Example pattern: Data Validation (e.g., SQL, HTML, XACML, etc.)

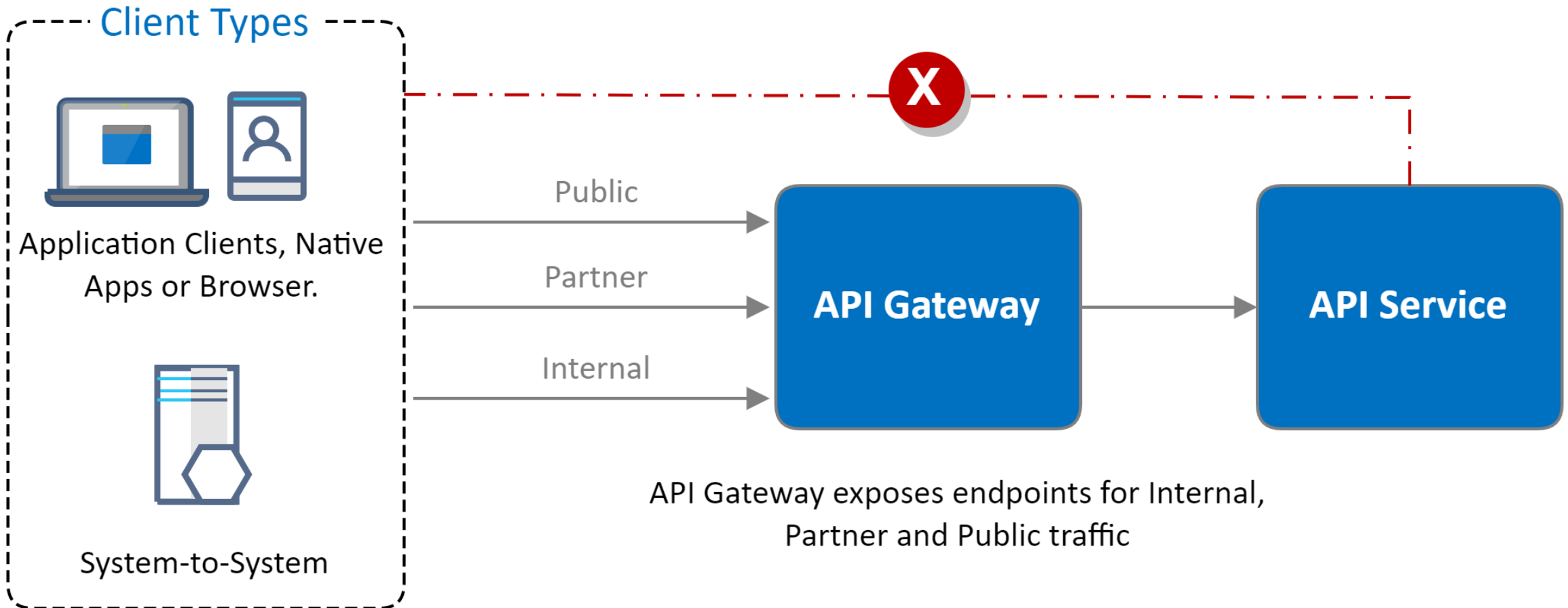
Image credit: [security pattern catalog](#)

Additional resource: [OWASP Input Validation Cheat Sheet](#)



Generic API gateway

Image credit: securitypatterns.io



I&A (Identification & Authentication)

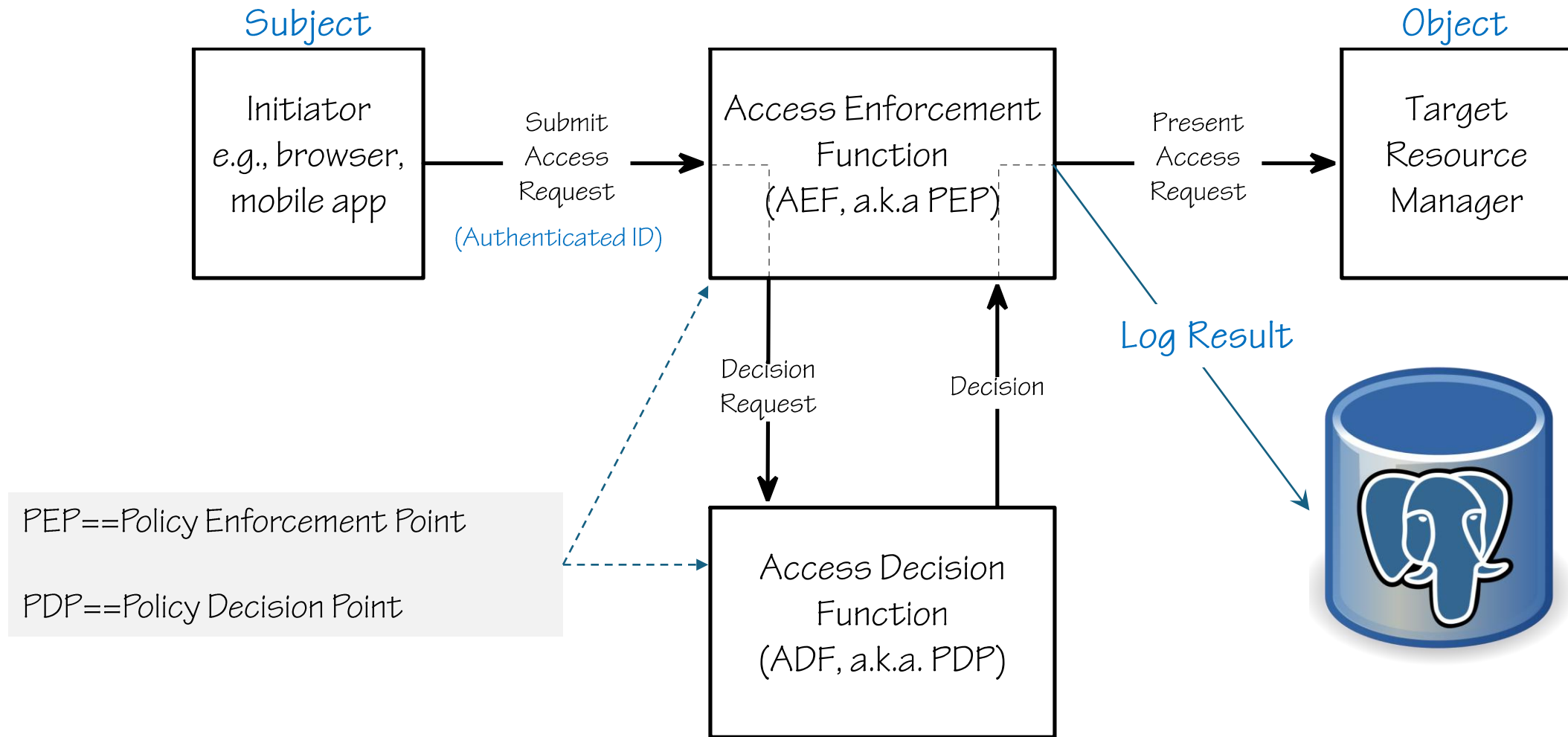
Additional resource: [OWASP Authentication Cheat Sheet](#)

- Identification: The ability to uniquely identify an entity (human, program, device, ...) in a system.
 - **Identity proofing** is the first step and is often challenging to do right.
- Authentication: The ability to prove that an asserted entity identity is genuinely what it claims to be.
- Authenticated identities have two primary uses:
 1. Input to access requests,
 2. Evidence w.r.t. non-repudiation of action.
- The rigor/robustness of I&A should be (but often is not) aligned with the assessed risk.

Authentication (AuthN) mini-taxonomy

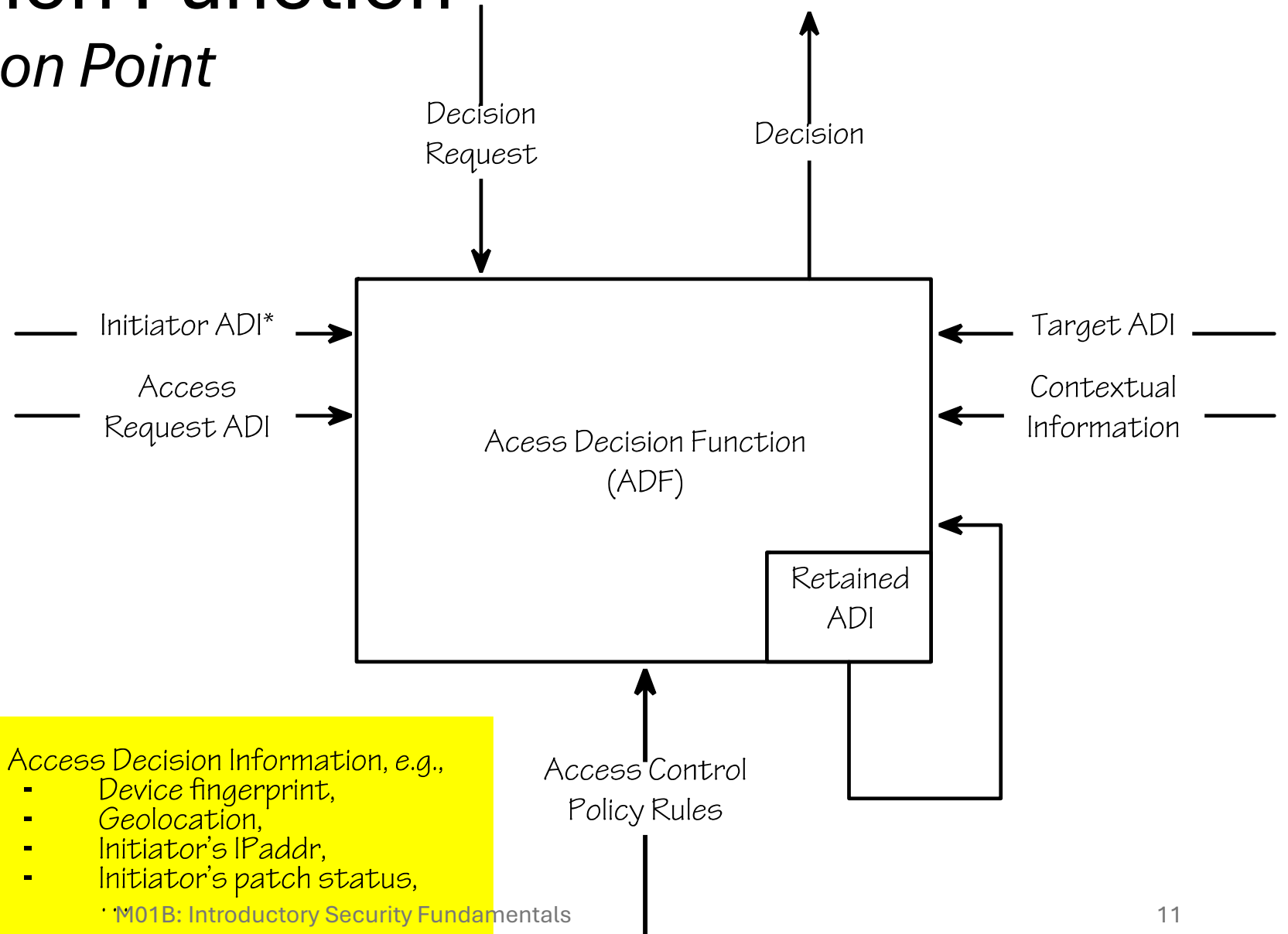
Type (weak-to-strong/low-to-high costs to deploy and maintain)	Examples (not comprehensive)
SFA (single-factor AuthN)	Something you know (userid+password)
	Something you have (e.g., RFID badge)
Passkeys	Your device(s) plus something you know/are
MFA (multi-factor AuthN)	UserId+password+SMS texted OTP (one-time password substitute)
	UserId+password+software hardware token (e.g., Duo Mobile)
	U.S.G. PIV (Personal Identity Verification) smart card: something you have (the card), something you know (method to unlock the card's functions, and something you are (biometrics))

ISO/IEC 10181-3 Access Control Model



Access Decision Function

a.k.a. Policy Decision Point



Least privilege [definitions]

Source: https://csrc.nist.gov/glossary/term/least_privilege

- A security principle that a system should restrict the access privileges of users (or processes acting on behalf of users) to the minimum necessary to accomplish assigned tasks.
 - [NIST SP 800-12 Rev. 1](#) under Least Privilege from CNSSI 4009
- The principle that a security architecture is designed so that each entity is granted the minimum system resources and authorizations that the entity needs to perform its function.
 - [NIST SP 800-53 Rev. 5](#)
- The principle that a security architecture is designed so that each entity is granted the minimum system authorizations and resources needed to perform its function.
 - [NIST SP 800-171r3](#)

Least privilege [continued]

- Historically, too much software development inherited the developers' privilege settings, e.g., too many Unix commands started with `root` (a.k.a. Admin) privilege, were subsequently abused and many things broke when trying to dial back their privileges to what they ought to be.
- Multiple hardware+software stacks have the idea of supervisor (privileged/root) and problem (user) execution states, but it must be properly used.
- Win95 has rudimentary support for user accounts, which is a problem since so much operational technology (OT) runs on Win95.
- When users/entities need higher privileges, Privileged Access Management (PAM) should be used/enforced.
- Least privilege must be top-of-mind for all parts of the secure software development lifecycle (sSDLC).

Secure defaults/paved roads

- One of the dumbest/laziest, still-in-existence problem is software that comes with default passwords and/or configuration values that can be (and are!) exploited by bad actors.
- Part of this is for the convenience of the development team (including testing).
- Part of this is because installation, configuration, scheduled maintenance (and emergency patching) was too-often excluded from the SDLC budget and plan.
- Fixes include forcing password changes (to good ones, with strong authentication the cherry on top), forcing “opt-in” selection during configuration for any features, functions, or services not absolutely required (e.g., open UDP/TCP ports), etc.

Providing *risk-appropriate* **availability**

- For most organizations and individuals, it's a matter of **when**, not **if** they will be affected by natural (e.g., tsunami) or malicious (e.g., ransomware, data breach, fraud, DOS/DDOS, etc.) events.
- **Continuity** and **response plans** need to be developed or purchased/rented and customized/outsourced.
- These plans must be iteratively tested and modified (“forever”).
- These plans must encompass *all* at-risk assets – which means “get your head out of the sand and know what you have!”
 - E.g., people, accounts, data bases, files, networks, cloud deployments, ...

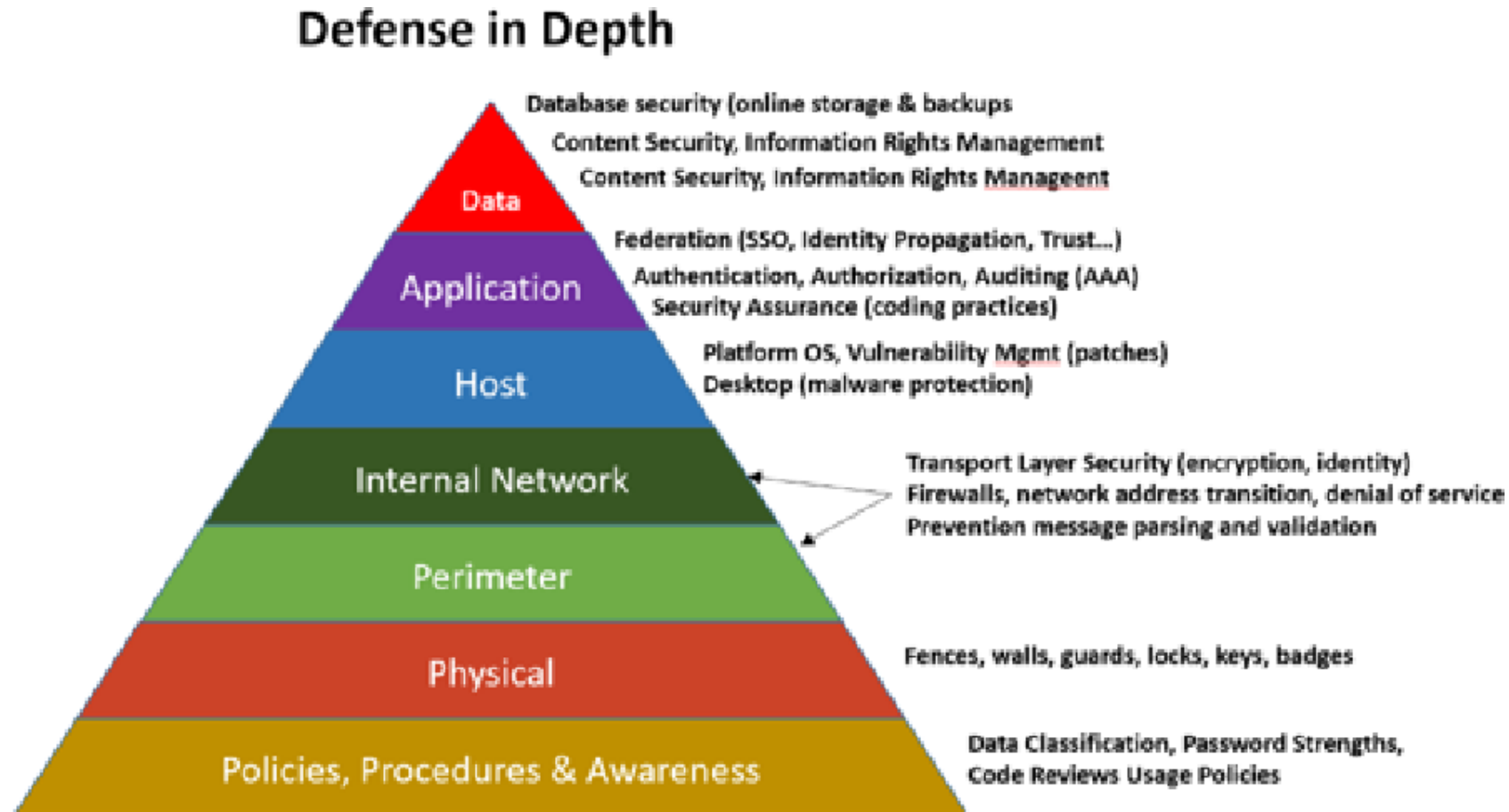
Defense-in-Depth (DiD) [definitions]

Source: [*NIST Glossary*](#)

- An information security strategy that integrates people, technology, and operations capabilities to establish variable barriers across multiple layers and missions of the organization.
- Information security strategy integrating people, technology, and operations capabilities to establish variable barriers across multiple layers and dimensions of the organization.
- **The application of multiple countermeasures in a layered or stepwise manner to achieve security objectives. The methodology involves layering heterogeneous security technologies in the common attack vectors to ensure that attacks missed by one technology are caught by another.**

DiD: *an* (not *the*) example

Image credit: [semanticscholar.org](https://www.semanticscholar.org)



Software supply chain security

Source: Crossley, C. Software Supply Chain Security

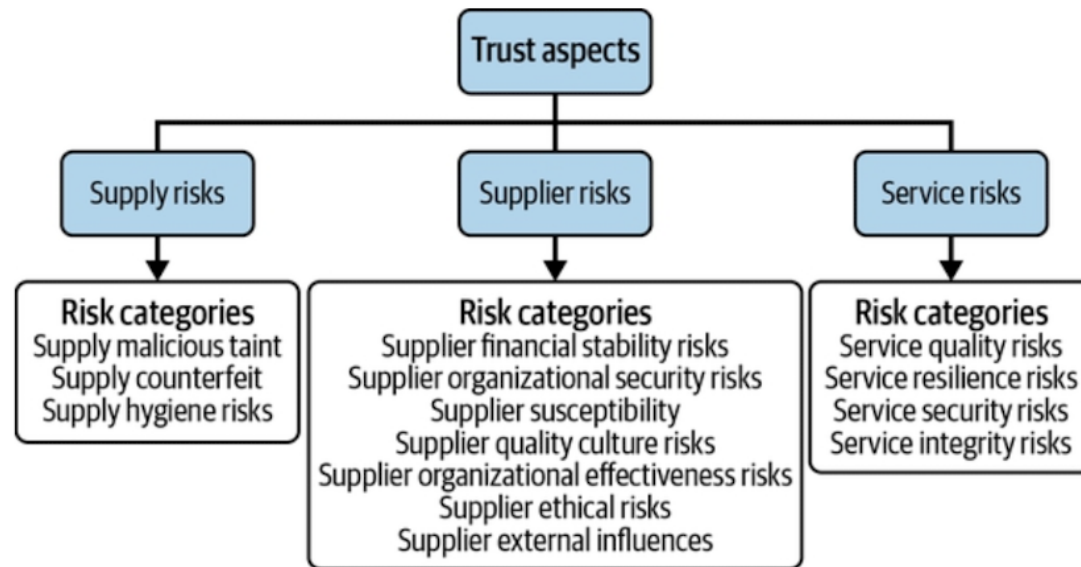


Fig. 2-13. MITRE System of Trust ([SoT](#))

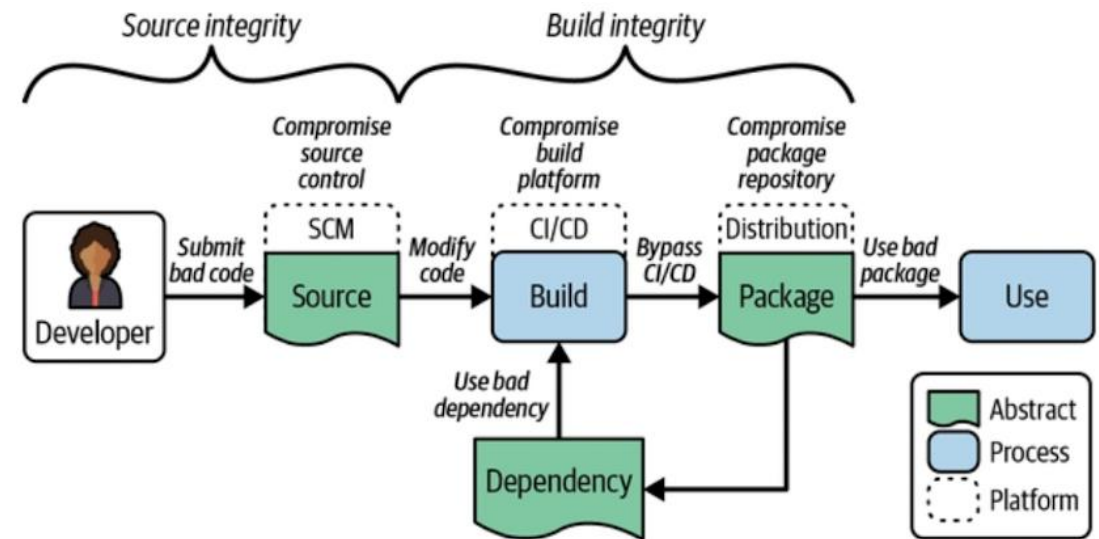


Fig. 5-2. Potential threats in the SDLC:
Google Supply-Chain Levels for Software
Architects ([SLSA](#))

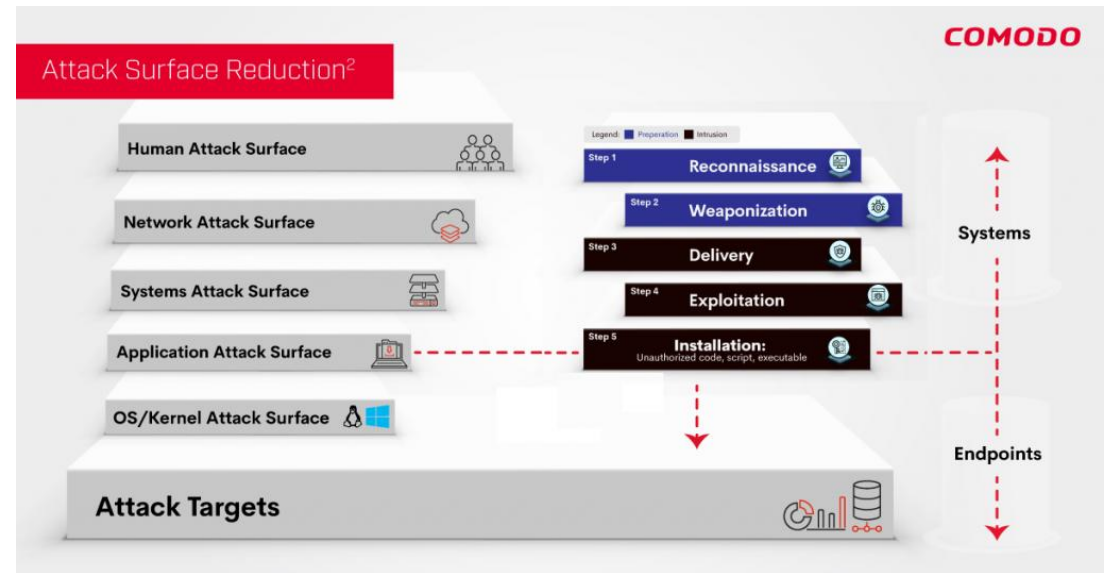
Security through obscurity

Related: IDEs that “scramble” executable code to make reverse-engineering harder

- In brief: this security “control” seeks to hide the mechanisms or details of a system to enhance its security.
- It may be a small part of a systems security controls, but it must never be the only control.
- Remember: hiding secrets is hard, and a motivated attacker will find a way to ferret them out or bypass them.
- Example: not broadcasting the SSID of a Wi-fi network.
- Tension: availability requirements bend system designs to compromise on security to enable automated restart/recovery (e.g., stash files).
 - Alternative for some scenarios: redundancy while humans handle recovery, or third-party managed operations.

Attack surface reduction (ASR)

- ASR is essential, and it will be covered in more depth later.
- The textbook's author discusses the elimination of “unused/dead code” that can be discovered through human review, static source code analysis tools and Unit Testing.
- Rigorous testing at all levels is required after code removal.
- Development/test instrumentation needs to be treated as unused code (i.e., excised, and retested) before release to users.



Credit: [Comodo.com](https://www.comodo.com)

Usable security

- Security controls are de facto resented.
- So, don't give users even more reasons (real and perceived) to resist, evade and ignore security controls (people, process & technology).

Fail closed/safe, then roll back

- The software you develop needs to be maniacally focused on transaction/data integrity.
- If the software deals with high-value assets, it must have an alternate mode of execution (refer to the oral example of the NYSE or other “break the glass” use cases), which requires more than software.
- Note that good software hygiene (e.g., make certain you always clear and free dynamically acquired storage) contributes to this security requirement.

Compliance, laws and regulations

Being in compliance ≠ being secure!

- Safety, security and privacy laws and regulations (L&R) exist because?
- L&R almost always lags the threat landscape.
- Most U.S. L&R doesn't cause enough pain to violators to affect bad behavior, and people suffer "incident fatigue."
- Entities covered by L&R lobby hard against them, but it's nearly the null set when it comes to entities demonstrating effective "self regulation."
- The better L&R regimes strive to stay current and fix discovered gaps.

Security and the SDLC

Organization, team and individual mileage may vary

- Security/privacy frameworks and assurance regimes.
- Threat modeling aligned with risk assessment – beware of “movie plot” use cases.
- Security SMEs, mentors, retained knowledge/expertise.
- The software forward compatibility “spectrum.”
- Software maintenance (patching) too often doesn’t get the “law of unintended consequences” treatment (resources) it needs.